Conceptual Architecture Report for Gemini CLI

Group 5: It Compiles (Sometimes)

Assignment 1

Friday, February 13, 2026

Group Leader: Edward Tanurkov - edward.tanurkov@queensu.ca

Presenter 1: Brian Constandes - 22bac1@queensu.ca

Presenter 2: Arham Hassan - 23cr8@queensu.ca

Ajwaad Khan - 19aak19@queensu.ca

Junoh Loy - 22DLT1@queensu.ca

Faizan Muhammad - 22mq34@queensu.ca

Abstract

Gemini CLI is an open-source, terminal-based AI agent that brings Google's Gemini model straight to the command line (Google, "Tutorials"). It is designed to integrate directly with the user's local environment, assisting with tasks such as coding, debugging, manipulating files and directories, and executing shell commands (Alateras). In addition to local system interaction, Gemini CLI allows the user to connect to external services and tools, such as searching the web and providing real-time information to its reasoning. As a result, Gemini CLI's architecture is highly complex and ideal for a detailed architectural analysis. The purpose of the project is to analyze, document, and evaluate the software architecture of Gemini CLI to gain a deep understanding of how the terminal-native AI tools system is structured and implemented. In the report, we discuss key architectural findings, including the CLI frontend, core backend, context manager, safety/approval subsystem, and tool execution layer. We show how they work simultaneously to provide live functionality while ensuring safe execution of user commands. The architecture manages user requests, tool execution, and context updates by merging modular pipelines and event-driven behaviour while also combining layered design with client-server interactions

Introduction

Gemini CLI is an open-source, terminal-based AI agent created by Taylor Mullen, a Software Engineer for Cloud AI at Google. This tool enables users to interact with Google's Gemini models directly from the command line. It was built to support everyday developers' workflows, including coding, debugging, handling files and directories, and running shell commands. Unlike other chatbots, Gemini CLI is agentic: it can reason, select tools, and carry out multi-step plans while providing transparency into its actions. Additionally, it has access to the web and other Google services to enhance its reasoning with real-time information.

Gemini CLI's hybrid architectural style integrates multiple subsystems, making it modular, extensible, and easy to maintain while supporting complex interactions with the user's local environment and external system services.

At a high level, Gemini CLI has six main subsystems. The CLI frontend manages all user interactions and serves as the main interface for accessing features. The Core Backend handles the system's core functions and contains all the logic for AI interaction, configuration management, and communication with external services. The Tool Execution Layer is a comprehensive set of features essential for processing requests across Gemini CLI components. The MCP Integration subsystem uses the same package as the tools to establish MCP connections to other platforms, enabling a chatbot like Gemini to communicate directly and use appropriate tools to streamline workflows. The Context Manager combines file-based

instructions and conversation history to support AI reasoning. Lastly, the Safety/Approval subsystem acts as a safeguard, controlling tool execution and system modifications.

The report begins by showing two use cases: the first is files and shell commands, and the second is web search and to-dos. Following the use cases, we present the sequence diagram that shows the interactions between subsystems. We then discuss the conceptual architecture and architectural style. After this section, we dive into Control and Data flow, followed by the developers' responsibilities, as well as other architectural styles we could have considered. Finally, we discuss what we have learned throughout the project and our conclusion.

Use Cases and Sequence Diagrams

Use Case 1 - Files and Shell Commands

A primary use case for Gemini CLI involves leveraging its file system and shell execution capabilities to perform complex development workflows directly from the command line. In this scenario, a user needs to refactor code across multiple files in a project directory and execute shell commands to test the changes. The workflow begins when the user issues a command to the Gemini CLI to modify specific functions across several source files, then runs unit tests to verify the refactoring was successful.

The process starts with the user entering a command that describes the desired refactoring task. The CLI Frontend (Subsystem 1) parses this input and passes it to the Core Backend (Subsystem 2), which constructs a structured request for the Gemini API. The backend then communicates with Gemini's AI model to determine which tools are needed to complete the task.

Based on the user's request, Gemini identifies that File System Tools and Execution Tools from the Tool Execution Layer (Subsystem 3) are required (Google, "Gemini CLI Tools"). The Context Manager (Subsystem 5) retrieves relevant information from GEMINI.md and includes conversation history to provide context for the AI's reasoning (Google, "Memory Tool"). Before any modifications occur, the Safety/Approval subsystem (Subsystem 6) presents the proposed file changes and shell commands to the user for explicit approval.

Once the user approves, the File System Tools read the target source files, perform the requested modifications, and write the updated content back to disk (Google, "Gemini CLI File System Tools"). The Execution Tools then construct and run shell commands to execute the test suite (Google, "Shell Tool"). Throughout this process, the Memory Tools can be invoked to save important information about the refactoring for future reference. The results are formatted by the Core Backend and displayed to the user via the CLI Frontend, including a summary of modified files, test results, and any relevant output from shell commands.

Below is a sequence diagram illustrating how all subsystems collaborate to execute this use case:

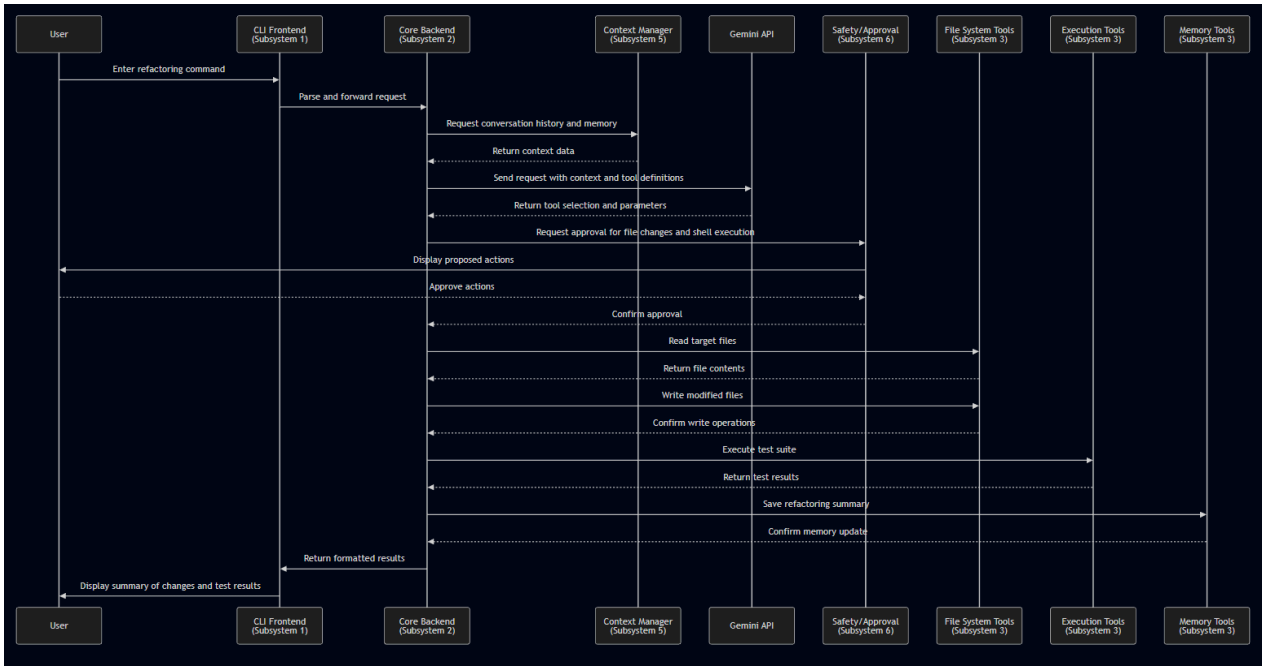

Figure 1. Use Case 1 Diagram. Dotted lines are responses, solid lines are requests.

Use Case 2 - Web Search and Todos

A second possible use case is using web search in conjunction with the Todos tooling to complete research tasks for any type of project (Google, "Web Search Tool"; Google, "Todo Tool"). The main overview would be for the user to perform a search query on a particular topic, and once complete, a Todo for beginning or completing this research is updated in the applicable Gemini CLI tooling. This Todo could be part of an overall workflow for a broader topic, like researching a lab report (main workflow) and needing information for a citation relevant to your discussion (Todo), for example.

The next page is a sequence diagram showing how every subsystem in the overall architecture works in cohesion when executing this use-case. We further identify and explain these subsystems in the next section, **Conceptual Architecture**. The use of **Subsystem 4 - MCP Integration** is not key to this solution, unless somehow users are able to, for example, use a search query to fix a GitHub workflow and make their code work, thereby updating a Todo that was related to fixing said workflow. As such, the subsystem is left as an optional sequence for this use case.

Conceptual Architecture

On the following page, we showcase the overall conceptual architecture of Gemini CLI. We have 6 major subsystems identified, each within their own white boxes as shown in the box-and-line diagram. They each match the 6 subsystems shown in the use-cases identified earlier. After the

next page, we will begin diving into deeper detail as to what each subsystem is, and what they do.

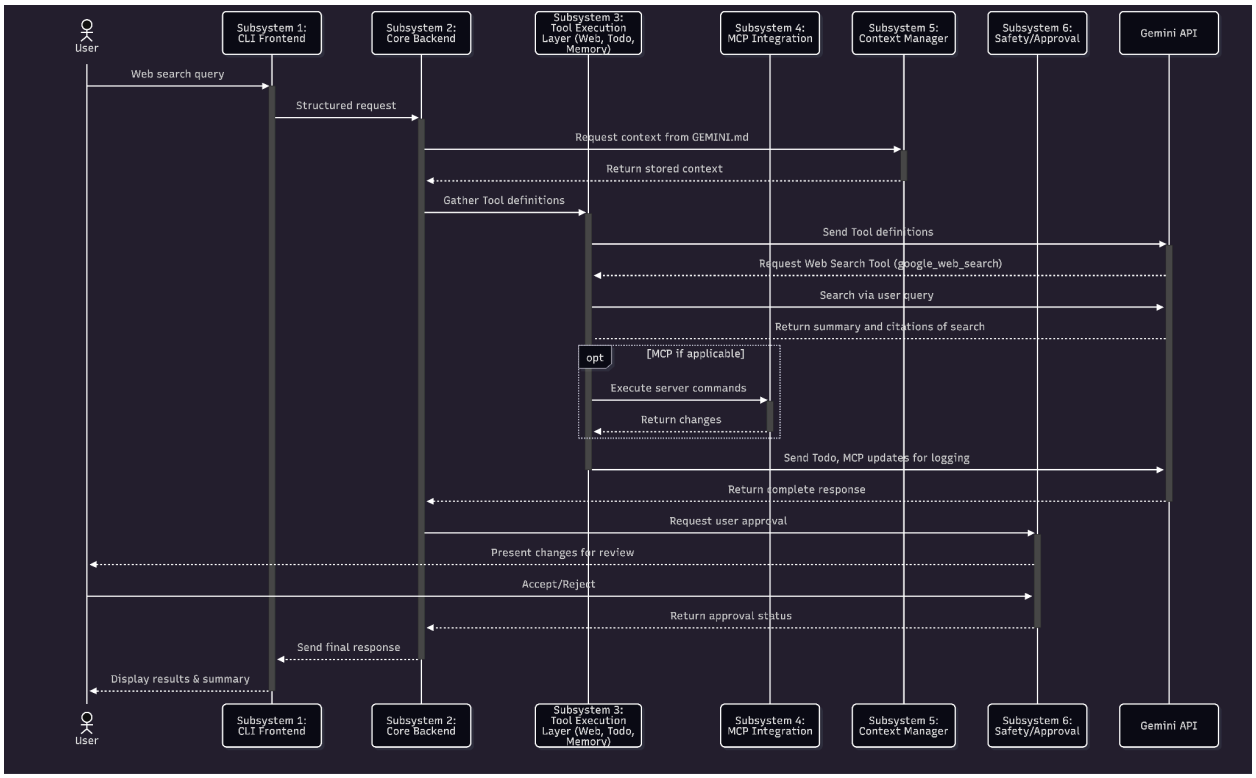

Figure 2. Use Case 2 Diagram. Dotted lines are responses, solid lines are requests.

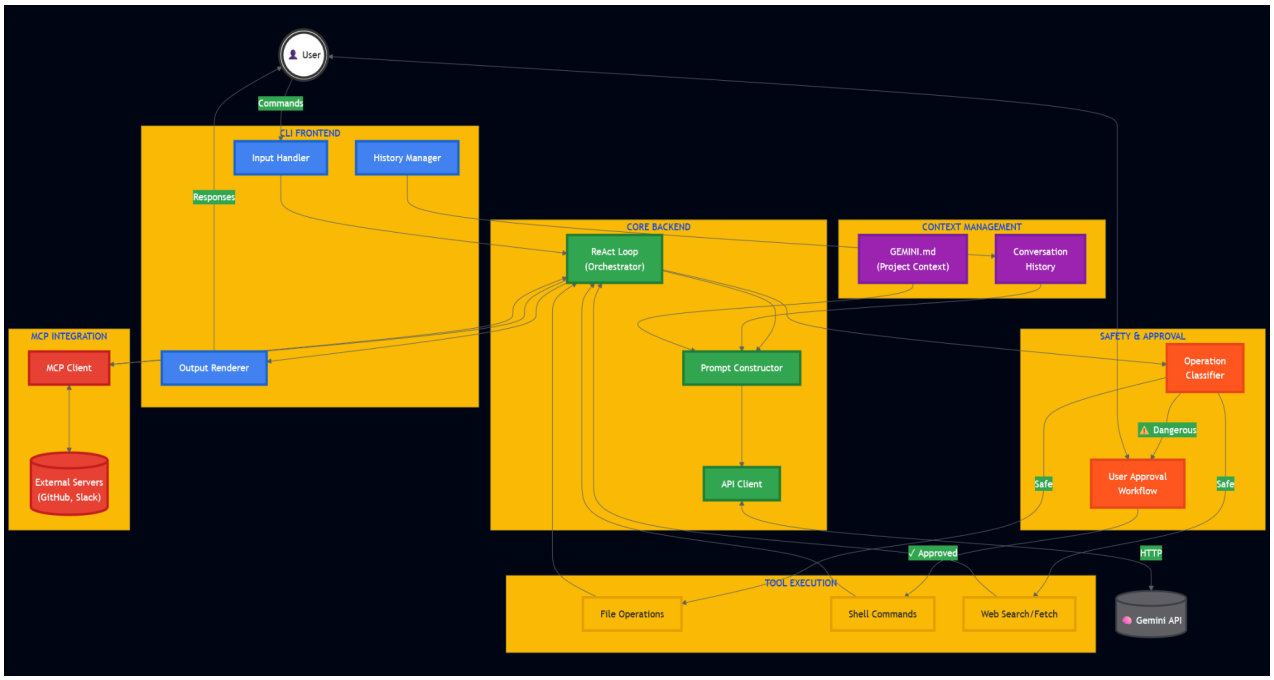

Figure 3: Gemini CLI Conceptual Architecture - Showing the six major subsystems and their runtime interactions.

Subsystem 1 - CLI Frontend

The CLI frontend subsystem in Gemini CLI handles all interactions between the user and the system. It serves as the main interface for accessing functionality. Its primary job is to understand the user's input from the command line and convert it into a structured request that the backend can process. This subsystem does not implement any business logic related to AI inference or API communication. By keeping user interactions separate, the CLI frontend ensures that changes in command syntax or user experience do not impact the backend logic.

One of its most important duties is command and argument parsing. When a user runs the Gemini CLI, the frontend interprets positional arguments, optional flags, and configuration options provided at runtime. The frontend also performs early validation by checking for missing arguments, invalid flag values, or conflicting options before executing the command. This validation helps prevent incorrect requests from reaching the backend and reduces errors while improving system reliability.

Another key responsibility is input acquisition and normalization. The frontend must handle various input methods common in command-line environments, including direct command arguments, piped input through standard input, and file-based input. The frontend gathers input from these sources and normalizes it into a consistent format, ensuring the backend receives a uniform representation of user input regardless of how it was provided.

The frontend also plays a crucial role in providing user feedback and documentation. It displays help messages, usage instructions, and command descriptions when requested or when it detects incorrect usage. In case of user errors, the frontend generates clear and actionable error messages. Additionally, the frontend uses appropriate exit codes to indicate success or failure, which is important for scripting and automation workflows.

Output formatting and presentation are also major responsibilities. Once a response is received from the backend, the frontend decides how to display that response in the terminal. This includes showing plain text output, supporting structured formats like JSON for machine-readable output, and handling streaming responses. The frontend must also differentiate between standard output and error output.

In summary, the CLI frontend subsystem acts as a thin, stateless, user-focused interface. Its responsibilities include parsing and validating user input, acquiring and normalizing data from multiple sources, providing clear feedback and documentation, formatting output for the

terminal, and managing execution flow. This clear division of responsibilities helps the Gemini CLI remain modular and easy to maintain while ensuring a smooth user experience.

Subsystem 2 - Core Backend

The core backend subsystem in Gemini CLI handles the system's key functions and contains all the logic related to AI interaction, configuration management, and communication with external services. In contrast to the CLI frontend, which emphasizes user interaction, the backend works independently of how commands are delivered. Its primary job is to receive structured requests from the frontend, process them according to system rules and settings, and send back clear responses. This separation keeps backend logic stable, even if the user interface or command syntax changes.

A key duty of the backend subsystem is managing configurations and environments. The backend must gather configuration values from various sources, such as environment variables, configuration files, and command-line defaults provided by the frontend (Google, "Gemini CLI Configuration"). These configurations may cover model selection, token limits, safety settings, and response formatting preferences. The backend verifies that configuration values are valid and applied consistently across requests. By centralizing this handling, the system prevents duplication and inconsistencies.

Authentication and authorization are also essential roles for the backend subsystem. The backend controls access to the Gemini API by managing credentials like API keys or Google Cloud authentication tokens. This includes locating credentials, checking their availability, and ensuring they are used securely during requests. The frontend is designed to remain unaware of these mechanisms, which helps protect sensitive information.

Another important task for the backend is request construction and validation. After receiving input from the CLI frontend, the backend converts this input into a structured request that meets the Gemini API specification. This process involves assembling prompt data, adding configuration parameters, and enforcing safety policies when needed. The backend also conducts deeper validation at this stage, ensuring that requests adhere to API constraints such as token limits.

The backend subsystem also oversees communication with the Gemini API. This includes sending requests, managing network communication, and handling retries or timeouts during temporary issues. In streaming scenarios, the backend handles partial responses and sends them to the frontend in a controlled manner. Response processing is another vital responsibility, where the backend extracts relevant information from raw API responses, deals with metadata and safety signals, and translates API-level errors into understandable results.

In summary, the core backend subsystem serves as the operational foundation of the Gemini CLI. By containing all business logic and external interactions, the backend provides a stable and reusable base that supports the CLI frontend while maintaining modularity, security, and ease of maintenance.

Subsystem 3 - Tool Execution Layer

As a big feature of the packages/core package, Gemini CLI Tools (packages/core/src/tools) are integral to how requests are processed by the components of Gemini CLI. The main groups of these tools are: File System Tools, Execution Tools, Web Tools, Memory Tools, and Todo Tools. Each group has numerous functions within them that allow Gemini CLI to perform various tasks on the user's computer, and according to the official docs these tasks include (but aren't limited to): accessing local information, executing shell commands, interacting with the web, taking actions, and grounding responses with real-time data (Google, "Gemini CLI Tools").

With all this functionality, tools at a high-level are invoked when a user executes a command, where the packages/core component of Gemini CLI sends the definitions of all the tools to Gemini. The model then chooses and executes the appropriate tool (if needed) and returns a formatted result for the user to accept or reject the changes. If the user accepts them (see **Subsystem 6 - Safety/Approval**), a summary of the changes will be shown in the CLI interface for clarity of what the tool execution performed.

Group 1: File System Tools

This is the tool component that enables file read/write capabilities on the user’s machine, only having access to the current working directory where Gemini CLI is launched, unless absolute paths are used to access external files. The functionality of this tool is for: reading, writing, listing, searching for, and modifying any files or directories via user commands in Gemini CLI. Users are able to specify certain limitations, such as which paths to follow, how many lines to change, how far into a file/directory Gemini should start searching from, etc. A good prompt engineer will be able to make very effective use of this tool, utilizing minimal tokens and speeding up file manipulation processes (Google, "Gemini CLI File System Tools").

Group 2: Execution Tools

This tool is used for shell scripting on the user’s machine. There is only one main function that this tool uses, but it has many different configurations which can perform a wide amount of tasks. A core part of this tool is that users may specify in a `settings.json` file which kind of shell commands are enabled and disabled as to protect their device (no one wants an accidental `rm -rf /` on their system…). Past all the customizability, the tool creates shell commands from a user request that will easily perform anything the users asks for (moving directories, deleting certain files, starting servers, etc.) with a simple query (Google, "Shell Tool").

Group 3: Web Tools

The Web Tools component is simplified to two particular sub-components: `web_fetch` and `google_web_search`. The web fetch sub-component is prompted to fetch requested information from up to 20 URLs, including summaries, key points, specific quotes, etc. Meanwhile the web search component simply creates a search query in Google and afterwards Gemini returns a summary of the search results along with citations (Google, "Web Fetch Tool"; Google, "Web Search Tool")..

Group 4: Memory Tools

Another component with only one function, but unlike the Execution Tool, this one is simpler in nature. It is used to save requested information in the memory portion of the context window for your Gemini model. The user sends in this information for Gemini to remember, and then Gemini CLI appends it to `GEMINI.md`, which is where Gemini retrieves memory before every query for recall (Google, "Memory Tool").

Group 5: Todo Tools

Finally, the Todo Tools are used in order to create progress tracking behaviour amongst the model for specific tasks the user wants Gemini to do. Essentially, it is used for the user to easily track which stage of processing a task or series of tasks the model is on, so that the user can ensure the model is following appropriate processing steps and is not straying off course. This is handy for anyone who uses agentic AI often in their workflows (Google, "Todo Tool").

Architectural note on Tools

A great part of the modularity that comes with separating these groups of Tools and the Tools from the rest of the Gemini CLI architecture is that it is much easier to integrate new changes to the Tools, since it is isolated from the rest of the framework. Thus if Google wants to add more functionality to Tools, the testing is streamlined and separate from other Gemini CLI components.

Subsystem 4 - MCP Integration

Gemini CLI allows developers to utilize their own MCP connections to other platforms (e.g. GitHub) so that a chatbot, such as Gemini, can communicate directly with the platforms and use appropriate tools to streamline workflows. Gemini CLI integrates MCP connections for this manner, heavily utilizing the same package that the Tools (packages/core/src/tools) use for server discovery and prompt execution. The documentation clearly outlines the architecture as having the following components: **Discovery Layer, Execution Layer**, and **Transport Mechanisms** (Google, "MCP Servers").

MCP Integration Setup

Users configure an MCP server in a `settings.json` file using a specific structure requested by Gemini CLI. Users can configure specific server names to include and exclude, along with specific commands Gemini CLI has access to, what environment variables are needed, the working directory, connection URL, and more (Google, "MCP Servers").

Discovery Layer

This layer initially goes through every server listed in your `settings.json` file to establish a connection. Once established, it is able to search and find specific tool definitions via MCP protocol that are listed in the server, and passing these definitions to Gemini to understand and validate them against Gemini's API (so they may be integrated as custom tools). After registering and fetching appropriate resources for these tools, this MCP server is now set up and users have the ability to use Gemini CLI on their server along with the tools integrated with it in an efficient manner (Google, "MCP Servers") .

Execution Layer

After a server is fully discovered by the Discovery Layer, Gemini CLI now has the ability to execute commands and tool usage. As such, this layer is responsible for maintaining the connection state to the MCP server. In a regular execution workflow, this layer first ensures that the tools and commands it uses against the server are allowed by the server. Once a command is validated, it then is able to execute any tools (whether they are Gemini's or the server's) to perform tasks and workflows on the server, using appropriate parameters from the definitions read in the Discover Layer. Finally, this layer then processes any responses so that they may be forwarded through the backend to the frontend for user display (Google, "MCP Servers").

Transport Mechanisms

While the first two layers are responsible for defining the connection creation and maintaining it, this layer defines what MCP protocol transport types are allowed for use. As such, this layer isn't really part of the architecture apart from validating that these transport types are the ones being supported and used. They are Stdio Transport, SSE Transport, and Streamable HTTP Transport (Google, "MCP Servers").

Subsystem 5 - Context Manager

Purpose

Gemini CLI needs to be able to see the right information at the correct time. The Context Manager exists to assemble that information into a context packet that is coherent for the Core Backend to send to Gemini. This subsystem is responsible for combining file-based instructions

and conversation history to support AI reasoning. As mentioned in Use Case 1, it retrieves relevant info from Gemini.md and includes conversation history before the request is sent to the model.

Context Aggregation

The Context Manager is able to collect and merge multiple context sources into a single bundle, including persistent instructions and user approved memory, conversation history, workspace references when needed, and tool outputs created during an agent loop.

Memory Persistence and Recall Integration

The system's memory tool appends user-approved memory into Gemini.md, and this file is retrieved prior to subsequent queries to support recall (Google, "Memory Tool"). The Context Manager is the main consumer of persistent memory and incorporates it into the context bundle delivered to the core backend.

Relevance Filtering and Packing

Since the context windows of the model are limited, it specifically chooses high-signal information and leaves out irrelevant and redundant context. The way it prioritizes things is by always including persistent instructions, most recent conversational turns, and adding file and tool information only when they are relevant to the current request.

Runtime Behaviour

At runtime, the Context Manager reads persistent instruction sources and retrieves and formats conversation history into a model-consumable structure. It then incorporates tool observations when available and returns the assembled context bundle to the Core Backend for request construction.

Interfaces and Interaction with Other Subsystems

The Core Backend requests current context bundle for prompt construction. The Tool Execution Layer supplies tool outputs that become a part of the context. Memory tool updates Gemini.md which later gets incorporated into future bundles. The CLI Frontend depends on the Context Manager through the Core Backend. Safety and approval constraints what new context becomes available to be included.

Subsystem 6 - Safety/Approval

The Safety/Approval subsystem in the Gemini CLI works as a protection between the execution of tools and system modifications. The Safety/Approval subsystem ensures that any unintended

or negative actions are not carried out without the user's active approval. Unlike the other subsystems that are concerned with command processing or data management, the Safety/Approval subsystem deals with risk management and user control.

This subsystem is a form of approval that acts as a safety check before any tool execution request alters the user's system. When the Core Backend determines that a tool must be used for an action, such as modifying a file or interacting with a web page, it must first pass through the Safety/Approval system. This is to ensure that nothing happens without the user's notice, regardless of how confident the AI model may be in its appropriateness.

The approval process works in a loop that shows actions to the user in human-readable format. Instead of showing the user API calls or system commands, the subsystem translates technical operations into clear descriptions of what will happen. This makes the approval process more accessible to users without deep technical knowledge while still providing enough detail for informed decision-making.

An important aspect of this subsystem involves categorizing actions by risk level. Not all operations require the same amount of caution. Reading files or searching directories are typically lower risk compared to deletion operations or shell commands that modify system state. The Safety/Approval subsystem applies different approval thresholds based on these risk assessments. Low-risk operations might receive automatic approval or simplified confirmation, while high-risk actions always require explicit user consent.

The subsystem also handles user preferences for approval. It gives the user the option to choose which operations need approval and which operations can be performed without approval. When users review pending actions, the Safety/Approval subsystem provides context about why each operation is necessary. Rather than presenting isolated commands, it connects actions back to the original user request.

The subsystem also keeps a log of all approvals made. The logs show what was being requested, whether it was approved or denied, and when the request was made. For operations that involve multiple steps, the subsystem is able to batch related actions into a single approval request. This batching reduces interruptions while still maintaining transparency of all the modifications. By keeping the users in the loop for consequential decisions, this subsystem makes the software more safe and trustworthy while ensuring users maintain control over their environment.

Architectural Style Discussion

The architecture of Gemini CLI is built upon multiple styles, rather than relying on a single pure style. This hybrid style allows the system to be modular, extensible, and easy to maintain while supporting complex interactions within the user's local environment and external services.

Layered Architecture

Gemini CLI mainly follows a layered architectural style where it divides responsibilities into distinct layers with clear roles (Alateras). At the top layer is the Command-Line Interface. This is where it handles user input and output and is responsible for parsing commands, displaying results, and managing user interaction. Below this is the core processing layer that interprets commands, coordinates tasks, and applies business logic. Underneath the core are supporting layers, such as file system access, execution tools, web tools, and memory management.

This structure improves separation of concerns by dividing responsibilities across the system, which makes the architecture easier to understand and maintain. Since each layer focuses on a specific role, changes can be made to the lower-level components without impacting the command-line interface. This design reflects the layered architectural style where each layer interacts only with the layers immediately adjacent to it.

Client–Server Characteristics

Gemini CLI also has characteristics of a client-server architecture. The CLI acts as a client that sends requests and the Gemini model and related AI services function as servers that process the requests and return responses. This separation allows computationally intensive tasks to be handled externally rather than on the user's local machine.

This style supports scalability and flexibility since updates to the AI model or server-side logic can occur independently of the client.

Pipe-and-Filter Elements

Some parts within Gemini CLI resemble a pipe-and-filter architecture. User commands pass through a sequence of processing stages such as parsing, validation, execution and response formatting, each performing a specific transformation before passing it along to the next stage.

This approach improves reusability and composability where individual filters can be modified or replaced without redesigning the entire system. It also makes the data flow through the system more predictably and easier to debug.

Event-Driven Elements

Gemini CLI includes aspects of an event-driven architecture. User actions, such as entering a command generates events that trigger specific handlers within the system. These handles then determine which tools or services should be activated.

This approach allows the system to remain responsive and extensible as new command types or tools can be added by introducing new event handlers rather than restructuring the entire application.

Control Flow & Data Flow

Control flow (sequence of execution)

The user enters a command or query in CLI, the CLI Frontend parses and forwards a structured request to Core Backend. The core Backend requests a context bundle from Context Manager, including Gemini.md and conversation history, the core backend constructs the Gemini API request using the user's request configuration and context bundle. The definitions for tools are provided to Gemini, enabling Gemini to pick tools as needed, The tools workflow can be described as definitions being sent and the model chooses a tool, results are then formatted back into the conversation loop. If tool use is required, the Core Backend routes the request to the Tool Execution Layer. The system requests user confirmation for impactful actions, systems requests user confirmation via Safety/Approval subsystems, use case 1 mentions this step. If approved, tools will execute but if they are denied, control returns to the Core Backend where it then formats the final output and returns it to the Frontend, which displays the results to the user.

Data flow (information exchange between subsystems)

User interaction begins as raw command text and any options are entered in the CLI. The CLI Frontend will then convert this into parsed or normalized request objects and forward them to the Core Backend, which becomes the system's primary representation of what the user is asking for. The Core Backend obtains a context bundle from the Context Manager. This bundle includes persistent instructions and user-approved memory which is stored in Gemini.md, along with conversation history that provides continuity and task grounding. The context bundle may also include relevant workspace information such as file snippets or summaries when necessary in order to ensure the model receives only the information that is needed for the current request. When tool use is available, additional data objects will flow between the model and the system. The model is provided with tool definitions from the Core Backend, which enables Gemini to choose appropriate tools as part of its reasoning process. If Gemini requests a tool, the tool call is then executed by the Tool Execution Layer. And the results that are produced then return to the Core Backend. The data flow will include approval related artifacts for high-impact actions. The system will then produce a proposed action summary describing the thing to be executed or changed and user approval/denial decisions are captured through the Safety/Approval subsystem. The decision is then returned to the Core Backend to determine whether an action will proceed otherwise the system has to revise its approach. The Core Backend will construct a final formatted response from the model output and any tool observations and then send it back to the CLI Frontend to display it to the user. The data flow can be summarized by the user input being captured and structured, context recorded for sensitive actions and the final response gets returned to CLI for presentation

Developer Responsibilities

Given that Gemini CLI is a highly-modularized system, the main and sub-components can easily be delegated to different developers. Developers working on the CLI Frontend can focus exclusively on command parsing, input validation, and terminal presentation (UI/UX) without needing deep knowledge of AI model integration or API communication. Similarly, Core Backend developers specialize in configuration management, authentication, and API interactions while remaining isolated from user interface concerns. The Tool Execution Layer is modularized in a way that allows its developers to work independently on the different tool groups with little cross-dependencies, allowing for simple testing and feature additions. For example, MCP Integration developers can extend third-party platform connections without modifying core functionality. Similar logic can be applied to all other Tool Execution Layer components.

Overall, the modularization of Gemini CLI reduces merge conflicts, allows teams to work in parallel, and ensures that changes in one subsystem rarely affects others. However, it still requires careful coordination where subsystems communicate to one-another, making documentation and inter-team communication still important for effective development.

Alternative Decisions

Several alternative architectural styles were considered. A pure microservices architecture could have separated subsystems such as Tools, Context, and Safety into independent services. While this would improve scalability, it would introduce unnecessary complexity for a local, single-user CLI tool. A fully monolithic design was another option. Although simpler to implement initially, it would reduce modularity and make future extensions harder to manage as the system grows. A more extensive plugin-based architecture could have maximized flexibility, but it would raise security and stability concerns, especially given the system's ability to execute shell commands and modify local files.

Finally, a purely event-driven architecture could increase decoupling but would add abstraction and debugging complexity for workflows that are mostly sequential and user-driven. Overall, the chosen hybrid architecture provides a balanced solution that supports modularity, safety, and maintainability without unnecessary complexity.

Derivation Process

Having mostly followed the official Gemini CLI documentation and visualizing the inter-dependency between the subsystems before even having officially made note or writing them out, we started thinking of the use-cases. We did initially write out where a few key areas were for the CLI and planned out how the group would delegate work amongst our members.

The key areas would be the subsystems we identified, especially the Tool Execution Layer upon which the most obvious use-case curation ideas came to light. Then after having planned out who would write which section of the report, the use-cases were created using just the names of the identified. We ended up writing a good chunk (about half) of the architectural details for each subsystem so as to garner more information on each subsystem, without yet creating connections between them unless there already were obvious connections. Given some more in-depth analysis having been done (and the conceptual architecture diagram not being started yet), we recognized that some use-cases could be created via specific configurations of Tools that developers can use at once in a way that makes sense. After some deliberation, we decided that the combination of Files and Execution Tools, and Web and Todo Tools made the most sense from a user's perspective. Then we made some links between how these Tools (and the other Tools) were inter-connected, and then inter-connected the Tool subsystem (3) to all the other subsystems, if applicable. The sequence diagrams' creation stemmed from all of these inter-subsystem connections given the particular use-cases we designed. From these sequence diagrams, the box-and-line conceptual architecture diagram was formed, alongside the now-completed subsystem descriptions (worked on further in parallel with the use-cases) that had enough detail as to see the inter-connectivity between each subsystem.

Lessons Learned

Throughout the development of this architectural analysis, we gained a deeper understanding of how complex systems can remain manageable through clear modularization and separation of concerns. Analyzing Gemini CLI showed us how breaking a system into well-defined subsystems such as the CLI Frontend, Core Backend, Tool Execution Layer, Context Manager, and Safety/Approval subsystem improves maintainability and scalability. We learned that architectural clarity is not only about functionality, but also about defining boundaries and responsibilities between components.

This project also reinforced the importance of hybrid architectural styles. Rather than strictly following a single design pattern, Gemini CLI combines layered, client-server, pipe-and-filter, and event-driven elements to achieve flexibility and responsiveness. This demonstrated that real-world systems often require architectural blending to meet practical requirements.

Additionally, we developed a stronger appreciation for safety and control in agentic AI systems. The Safety/Approval subsystem highlighted how architectural decisions directly impact user trust and system reliability. We learned that when software has the ability to modify files or execute commands, architectural safeguards are just as important as performance and usability. Overall, this project strengthened our ability to critically evaluate software architecture, analyze subsystem interactions, and justify design decisions using architectural principles.

Conclusion

Gemini CLI represents a modern example of a terminal-native AI system built using a hybrid and modular architecture. Through this analysis, we examined its major subsystems, including the CLI Frontend, Core Backend, Tool Execution Layer, MCP Integration, Context Manager, and Safety/Approval subsystem. Each subsystem plays a distinct role while working together to provide intelligent, safe, and interactive functionality.

The architecture balances layered structure, client-server interaction, and event-driven behavior to support both local system execution and external AI services. Its modular design allows for extensibility and maintainability while ensuring that safety and user control remain central to its operation. By studying Gemini CLI, we gained insight into how complex AI-powered tools can be structured to remain organized, secure, and adaptable. The system demonstrates how thoughtful architectural decisions enable advanced capabilities while preserving clarity, control, and long-term sustainability.

AI Collaboration Report

Our group opted for using Claude's free-tier Sonnet 4.5 model with a current training cutoff of January 2025 (Anthropic). We came up with Claude through online research across Reddit forums, and technology reviews on which public LLM model is currently superior in terms of software architecture and system design. We compared it to OpenAI's ChatGPT, Meta's Llama, Google's Gemini (ironically), and X's Grok. We immediately eliminated Grok and Llama from our list due to their inability to perform well in general compared to other LLMs. Afterwards, we found that ChatGPT is better for overall brainstorming and Gemini is better for longer-context situations, and finally Sonnet is best for coding problems and system design. This was a no-brainer for our group, since we would need the majority of our help with creating the diagrams and brainstorming around the design for these diagrams. We had three main use-cases:

- Help generate high-level plan of report structure, later modified by group members to add anything missing, remove anything unnecessary, flip structure around to something that makes more sense
    - Assigned so that we don't spend too much time organizing the entire report structure when our AI teammate can perform these logistics exponentially faster given the rubric, detailed document, needing minimal changes from our end
- Help identify subsystems within Gemini CLI
    - Scaned through the docs itself much faster than we can and provided preliminary subsystem ideas, while maintaining rubric, detailed document context
- Help create plan for use case 1 and 2 diagram, which were later heavily modified to simplify and fit report needs

    - With what descriptions we had in the report according to the **Derivation Process**, it was able to take in all of our context and brainstorm and create preliminary diagrams, which we then heavily changed and tuned to our needs
- Help brainstorm how to write certain parts of our essay
    - Given Sonnet's superior system design thought processes, we thought it would be able to generate feasible ideas on what to write initially. Some members either asked it to write more and then re-adjusted the writing to fit their style. We only used the "write more" portion on sections we deemed less important and were essentially summarizing parts of the essay (e.g. **Developer Responsibilities**)
- Help shortening the report (was a bit too long)
    - Helps easily identify portions which could be simplified given context based on reading the rest of the report

The entire group was responsible for using Sonnet for their personal needs (so as to not run out of tokens day-by-day), and we all followed a similar prompting practice. This practice was simply adding in relevant documents/context we needed to help solve a specific problem, getting the initial response, and then curating our follow-up responses to dig deeper into the solution or to ask why a certain procedure was followed. We also smartly prompted it not to write out entire sections of a brainstorming document again if we just wanted to change one thing, so as to save token usage for the day. For example, while generating the report structure it initially gave us a few hallucinated sections that were not needed upon our personal further inspection of the rubric. Thus we asked why they were added and Sonnet typically rethought the plan and removed the section, slightly readjusting the layout to fit better. We did this until we were satisfied and then rearranged the sections a bit ourselves so that we can make everything flow better in our view.

We always proof-read what Sonnet gave to us, especially while working on the diagrams it helped us create. The original diagrams were way too complicated and involved too many calls between classes (the sequence diagrams), so we had to clean up the diagrams and simplify them greatly while still maintaining appropriate context and knowledge pertaining to the course material. Any ideas it generated for Gemini CLI subsystems were immediately fact-checked within the documentation to ensure they were legitimate subsystems and were actually relevant. And for brainstorming, we already gave an example of how we ensured quality on that above.

We estimate the AI teammate did an equal split of work for us given that we did the bulk of the research and writing, while it helped mostly with ideation and getting the ball rolling (the more time-consuming and planning-oriented part that needed lots and lots of context). As such, we give it 1/7 (~15%) or a bit higher as an estimate of its contributions to the project, with the uses outlined above

Sonnet's addition saved a lot of time compared to the amount of prompt engineering and validation that was needed. We got rid of the long-winded planning that would take a long time to even get in the right places, but needing very little engineering from us and simply needing

proof-reading and tweaks. There were no disagreements or challenges within the group regarding its use, and we are very happy with how much it was able to help us. We appreciate the simplicity and effectiveness that Sonnet brought and are interested in continuing to utilize LLMs for brainstorming and initial planning purposes, especially when enormous amounts of information need to be consolidated in order to create an effective plan to create a solution.

References

Alateras, Jim. "Unpacking the Gemini CLI: A High-Level Architectural Overview." *Medium*, 8 July 2025, medium.com/@jalateras/unpacking-the-gemini-cli-a-high-level-architectural-overview-99212f6780e7.

Anthropic. "Anthropic's Transparency Hub." *Anthropic.com*, 2025, www.anthropic.com/transparency.

Google. "Gemini CLI Configuration." *Gemini CLI*, 2026, geminicli.com/docs/get-started/configuration/.

---. "Gemini CLI File System Tools." *Gemini CLI*, 2026, geminicli.com/docs/tools/file-system/.

---. "Gemini CLI Tools." *Gemini CLI*, 2026, geminicli.com/docs/tools/.

---. "MCP Servers with the Gemini CLI." *Gemini CLI*, 2025, geminicli.com/docs/tools/mcp-server/#2-tool-discovery.

---. "Memory Tool (`Save_memory`)." *Gemini CLI*, 2026, geminicli.com/docs/tools/memory/.

---. "Shell Tool (`Run_shell_command`)." *Gemini CLI*, 2026, geminicli.com/docs/tools/shell/.

---. "Todo Tool (`Write_todos`)." *Gemini CLI*, 2026, geminicli.com/docs/tools/todos/.

---. "Tutorials." *Gemini CLI*, 2026, geminicli.com/docs/cli/tutorials/.

---. "Web Fetch Tool (`Web_fetch`)." *Gemini CLI*, 2026, geminicli.com/docs/tools/web-fetch/.

---. "Web Search Tool (`Google_web_search`)." *Gemini CLI*, 2026, geminicli.com/docs/tools/web-search/.